Software Engineering

AICS 203

Unit I:

Introduction to Software Engineering

Kathmandu University
BTech AI (Second Year /Second Semester)
April 2026

Software Engineering

- **Some realities:**
  - *a concerted effort should be made to understand the problem before a software solution is developed*
  - *design becomes a pivotal activity*
  - *software should exhibit high quality*
  - *software should be maintainable*

Software Engineering History

- Computers were invented in the 1940's
- Then - computing programming languages were invented
- Eventually - program language training was developed
- However, training was unable to provide sufficient methods & techniques for developing large reliable systems on time & within budget
- By the late 1960's, digital computers were less than 25 years old and already facing a software crisis
- Software Engineering term first emerged as title of a 1968 NATO conference [1]

Software Development Statistics

- 1979 General Accounting Office Report [2]
  - 50% + of contracts had cost overruns
  - 60% + of contracts had schedule overruns
  - 45% + of software contracted for could not be used
  - 29% + of software was paid for and never delivered
  - 22% + of software contracted for had to be reworked/modified to be used

Software Development Statistics

- **Other Studies**
  - 1982 - Tom DeMarco
    - » 25% of large systems development projects never finished
  - 1991 Capers Jones Study
    - » Average Management Information System* project is 1 year late and 100% over budget
- **Software crisis stubbornly persists**

* Management Information System - a system for providing information to support organizational activities and management functions.

Software Costs

- Software costs often dominate computer system costs. The costs of software on a PC are often greater than the hardware cost.
- Software costs more to maintain than it does to develop. For systems with a long life, maintenance costs may be several times the development costs.
- Software engineering is concerned with cost-effective software development.

Software Project Failure

- ***Increasing system complexity***
  - As new software engineering techniques help us to build larger, more complex systems, the demands change. Systems have to be built and delivered more quickly; larger, even more complex systems are required; systems have to have new capabilities that were previously thought to be impossible.
- ***Failure to use software engineering methods***
  - It is fairly easy to write computer programs without using software engineering methods and techniques. Many companies have drifted into software development as their products and services have evolved. They do not use software engineering methods in their everyday work. Consequently, their software is often more expensive and less reliable than it should be.

Software Engineering

- **What is it?**

  Software engineering encompasses a process, a collection of methods (practice) and an array of tools that allow professionals to build high-quality computer software.

- **Who does it?**

  Software engineers apply the software engineering process.

Software Engineering

- **Why is it important?**

  Software engineering is important because it enables us to build complex systems in a timely manner and with high quality. It imposes discipline to work that can become quite chaotic, but it also allows the people who build computer software to adapt their approach in a manner that best suits their needs.

Software Engineering

- **What are the steps?**

  You build computer software like you build any successful product, by applying an agile, adaptable process that leads to a high-quality result that meets the needs of the people who will use the product. You apply a software engineering approach.

Software Engineering

- **What is the work product?**

  From the point of view of a software engineer, the work product is the set of programs, content (data), and other work products that are computer software.

  But from the user's viewpoint, the work product is the resultant information that somehow makes the user's world better.

Frequently asked questions about Software Engineering

Question	Answer
What is **software?**	Computer programs and associated documentation. Software products may be developed for a particular customer or may be developed for a general market.
What are the **attributes** of **good software?**	Good software should deliver the required functionality and performance to the user and should be maintainable, dependable and usable.
What is **software engineering?**	Software engineering is an engineering discipline that is concerned with all aspects of software production.
What are the **fundamental software engineering activities?**	Software specification, software development, software validation and software evolution.
What is the **difference** between **software engineering** and **computer science?**	Computer science focuses on theory and fundamentals; software engineering is concerned with the practicalities of developing and delivering useful software.
What is the **difference** between **software engineering** and **system engineering?**	System engineering is concerned with all aspects of computer-based systems development including hardware, software and process engineering. Software engineering is part of this more general process.

Frequently asked questions about Software Engineering

Question	Answer
What are the key **challenges** facing software engineering?	Coping with increasing diversity, demands for reduced delivery times and developing trustworthy software.
What are the **costs** of **software engineering?**	Roughly 60% of software costs are development costs, 40% are testing costs. For custom software, evolution costs often exceed development costs.
What are the best **software engineering techniques and methods**?	While all software projects have to be professionally managed and developed, different techniques are appropriate for different types of system. For example, games should always be developed using a series of prototypes whereas safety critical control systems require a complete and analyzable specification to be developed. You can't, therefore, say that one method is better than another.
What differences has the **web** made to software engineering?	The web has led to the availability of software services and the possibility of developing highly distributed service-based systems. Web-based systems development has led to important advances in programming languages and software reuse.

Software Products

- **Generic products**
  - Stand-alone systems that are marketed and sold to any customer who wishes to buy them.
  - Examples – PC software such as graphics programs, project management tools; CAD software; software for specific markets such as appointments systems for dentists.
- **Customized products**
  - Software that is commissioned by a specific customer to meet their own needs.
  - Examples – embedded control systems, air traffic control software, traffic monitoring systems.

Product Specification

- **Generic products**
  - The specification of what the software should do is owned by the software developer and decisions on software change are made by the developer.
- **Customized products**
  - The specification of what the software should do is owned by the customer for the software and they make decisions on software changes that are required.

Essential attributes of good Software

Product characteristic	Description
Maintainability	Software should be written in such a way so that it can evolve to meet the changing needs of customers. This is a critical attribute because software change is an inevitable requirement of a changing business environment.
Dependability and security	Software dependability includes a range of characteristics including reliability, security and safety. Dependable software should not cause physical or economic damage in the event of system failure. Malicious users should not be able to access or damage the system.
Efficiency	Software should not make wasteful use of system resources such as memory and processor cycles. Efficiency therefore includes responsiveness, processing time, memory utilisation, etc.
Acceptability	Software must be acceptable to the type of users for which it is designed. This means that it must be understandable, usable and compatible with other systems that they use.

Software Engineering

- Software engineering is an engineering discipline that is concerned with all aspects of software production from the early stages of system specification through to maintaining the system after it has gone into use.
  - Engineering discipline
    - Using appropriate theories and methods to solve problems bearing in mind organizational and financial constraints.
  - All aspects of software production
    - Not just technical process of development. Also project management and the development of tools, methods etc. to support software production.

Importance of Software Engineering

- More and more, individuals and society rely on advanced software systems. We need to be able to produce reliable and trustworthy systems economically and quickly.
- It is usually cheaper, in the long run, to use software engineering methods and techniques for software systems rather than just write the programs as if it was a personal programming project. For most types of system, the majority of costs are the costs of changing the software after it has gone into use.

Software Process activities

- **Software specification**, where customers and engineers define the software that is to be produced and the constraints on its operation.
- **Software development**, where the software is designed and programmed.
- **Software validation**, where the software is checked to ensure that it is what the customer requires.
- **Software evolution**, where the software is modified to reflect changing customer and market requirements.

General issues that affect Software

- **Heterogeneity**
  - Increasingly, systems are required to operate as distributed systems across networks that include different types of computer and mobile devices.
- **Business and social change**
  - Business and society are changing incredibly quickly as emerging economies develop and new technologies become available. They need to be able to change their existing software and to rapidly develop new software.

General issues that affect Software

- **Security and trust**
  - As software is intertwined with all aspects of our lives, it is essential that we can trust that software.
- **Scale**
  - Software has to be developed across a very wide range of scales, from very small embedded systems in portable or wearable devices through to Internet-scale, cloud-based systems that serve a global community.

Software Engineering diversity

- There are many different types of software system and there is no universal set of software techniques that is applicable to all of these.
- The software engineering methods and tools used depend on the type of application being developed, the requirements of the customer and the background of the development team.

Application types

- **Stand-alone applications**
  - These are application systems that run on a local computer, such as a PC. They include all necessary functionality and do not need to be connected to a network.
- **Interactive transaction-based applications**
  - Applications that execute on a remote computer and are accessed by users from their own PCs or terminals. These include web applications such as e-commerce applications.
- **Embedded control systems**
  - These are software control systems that control and manage hardware devices. Numerically, there are probably more embedded systems than any other type of system.

Application types

- **Batch processing systems**
  - These are business systems that are designed to process data in large batches. They process large numbers of individual inputs to create corresponding outputs.
- **Entertainment systems**
  - These are systems that are primarily for personal use and which are intended to entertain the user.
- **Systems for modeling and simulation**
  - These are systems that are developed by scientists and engineers to model physical processes or situations, which include many, separate, interacting objects.

Application types

- **Data collection systems**
  - These are systems that collect data from their environment using a set of sensors and send that data to other systems for processing.
- **Systems of systems**
  - These are systems that are composed of a number of other software systems.

Software Applications

- system software
- application software
- engineering/scientific software
- embedded software
- product-line software
- WebApps (Web applications)
- AI software

Software- New Categories

- Open world computing—pervasive, distributed computing
- Ubiquitous computing—wireless networks
- Netsourcing—the Web as a computing engine
- Open source—”free” source code open to the computing community (a blessing, but also a potential curse!)
- Also …
  - Data mining
  - Grid computing
  - Cognitive machines
  - Software for nanotechnologies

Characteristics of WebApps - I

- Network intensiveness. A WebApp resides on a network and must serve the needs of a diverse community of clients.
- Concurrency. A large number of users may access the WebApp at one time.
- Unpredictable load. The number of users of the WebApp may vary by orders of magnitude from day to day.
- Performance. If a WebApp user must wait too long (for access, for server-side processing, for client-side formatting and display), he or she may decide to go elsewhere.
- Availability. Although expectation of 100 percent availability is unreasonable, users of popular WebApps often demand access on a "24/7/365" basis.

Characteristics of WebApps - II

- Data driven. The primary function of many WebApps is to use hypermedia to present text, graphics, audio, and video content to the end-user.
- Content sensitive. The quality and aesthetic nature of content remains an important determinant of the quality of a WebApp.
- Continuous evolution. Unlike conventional application software that evolves over a series of planned, chronologically-spaced releases, Web applications evolve continuously.
- Immediacy. Although *immediacy*—the compelling need to get software to market quickly—is a characteristic of many application domains, WebApps often exhibit a time to market that can be a matter of a few days or weeks.
- Security. Because WebApps are available via network access, it is difficult, if not impossible, to limit the population of end-users who may access the application.
- Aesthetics. An undeniable part of the appeal of a WebApp is its look and feel.

Software Engineering Fundamentals

- Some fundamental principles apply to all types of software system, irrespective of the development techniques used:
  - Systems should be developed using a managed and understood development process. Of course, different processes are used for different types of software.
  - Dependability and performance are important for all types of system.
  - Understanding and managing the software specification and requirements (what the software should do) are important.
  - Where appropriate, you should reuse software that has already been developed rather than write new software.

Internet software engineering

- The Web is now a platform for running application and organizations are increasingly developing web-based systems rather than local systems.
- Web services allow application functionality to be accessed over the web.
- Cloud computing is an approach to the provision of computer services where applications run remotely on the ‘cloud’.
  - Users do not buy software buy pay according to use.

Web-based software engineering

- Web-based systems are complex distributed systems but the fundamental principles of software engineering discussed previously are as applicable to them as they are to any other types of system.
- The fundamental ideas of software engineering apply to web-based software in the same way that they apply to other types of software system.

Web software engineering

- **Software reuse**
  - Software reuse is the dominant approach for constructing web-based systems. When building these systems, you think about how you can assemble them from pre-existing software components and systems.
- **Incremental and agile development**
  - Web-based systems should be developed and delivered incrementally. It is now generally recognized that it is impractical to specify all the requirements for such systems in advance.

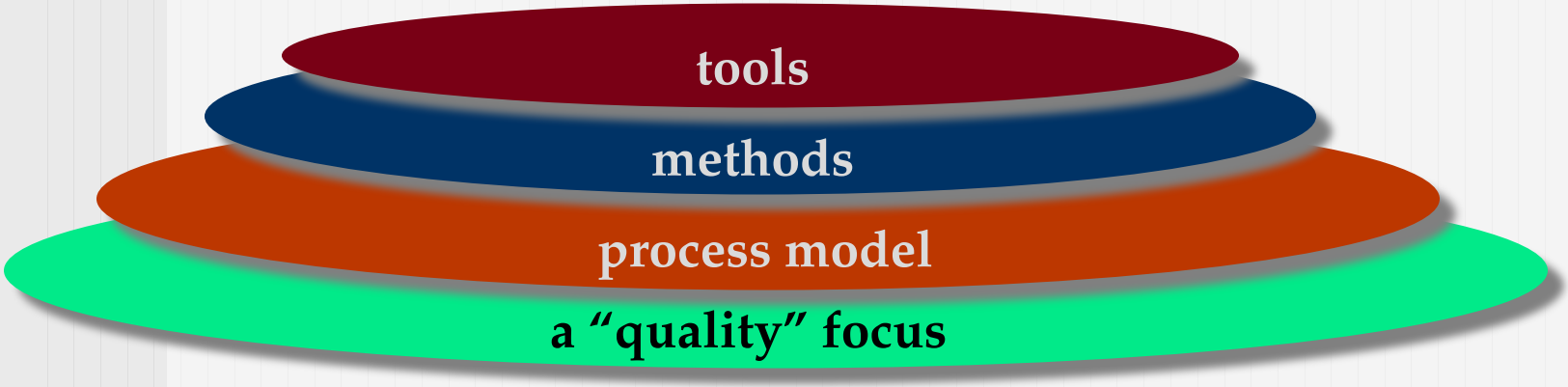
Web software engineering

- **Service-oriented systems**
  - Software may be implemented using service-oriented software engineering, where the software components are stand-alone web services.
- **Rich interfaces**
  - Interface development technologies such as AJAX and HTML5 have emerged that support the creation of rich interfaces within a web browser.

Software Engineering

- The IEEE definition:
  - *Software Engineering:*
  - *(1) The application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software; that is, the application of engineering to software.*
  - *(2) The study of approaches as in (1).*

A Layered Technology



Software Engineering

The Quality

- Any engineering approach (including software engineering) must rest on an organizational commitment to quality.
- Quality management is a continuous process improvement culture, and it is this culture that ultimately leads to the development of increasingly more effective approaches to software engineering. The bedrock that supports software engineering is a quality focus.

Process Model

- The foundation for software engineering is the *process* layer.
- The software engineering process is the glue that holds the technology layers together and enables rational and timely development of computer software.
- Process defines a framework that must be established for effective delivery of software engineering technology.
- The software process forms the basis for management control of software projects and establishes the context in which technical methods are applied, work products (models, documents, data, reports, forms, etc.) are produced, milestones are established, quality is ensured, and change is properly managed

Methods

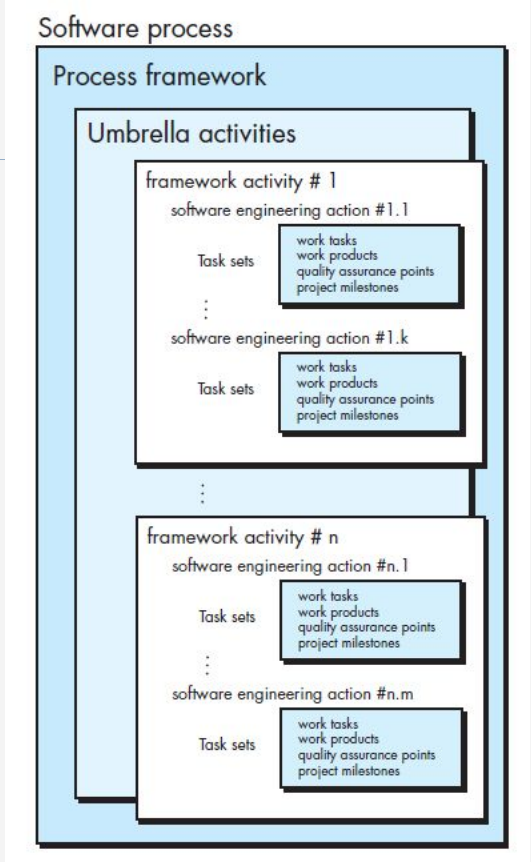
- Software engineering *methods* provide the technical how-to's for building software.
- Methods encompass a broad array of tasks that include communication, requirements analysis, design modeling, program construction, testing, and support.
- Software engineering methods rely on a set of basic principles that govern each area of the technology and include modeling activities and other descriptive techniques.

Tools

- Software engineering *tools* provide automated or semi-automated support for the process and the methods.
- When tools are integrated, information created by one tool can be used by another, a system for the support of software development, called *computer-aided software engineering*, is established.

A Process Framework

A process framework establishes the foundation for a complete software engineering process by identifying a small number of framework activities that are applicable to all software projects, regardless of their size or complexity. In addition, the process framework encompasses a set of umbrella activities that are applicable across the entire software process.



The Software Process

- A *process* is a collection of activities, actions, and tasks that are performed when some work product is to be created.
- An *activity* strives to achieve a broad objective (e.g., communication with stakeholders) and is applied regardless of the application domain, size of the project, complexity of the effort, or degree of rigor with which software engineering is to be applied.
- An *action* (e.g., architectural design) encompasses a set of tasks that produce a major work product (e.g., an architectural model).
- A *task* focuses on a small, but well-defined objective (e.g., conducting a unit test) that produces a tangible outcome.

Framework Activities

A generic process framework for software engineering encompasses five activities

1. Communication
2. Planning
3. Modeling
    a. Analysis of requirements
    b. Design
4. Construction
    a. Code generation
    b. Testing
5. Deployment

Umbrella Activities

- **Software project tracking and control**

  allows the software team to assess progress against the project plan and take any necessary action to maintain the schedule.

- **Risk management**

  assesses risks that may affect the outcome of the project or the quality of the product

- **Software quality assurance**

  defines and conducts the activities required to ensure software quality.

- **Technical reviews**

  assess software engineering work products in an effort to uncover and remove errors before they are propagated to the next activity.

Umbrella Activities

- **Measurement**

  defines and collects process, project, and product measures that assist the team in delivering software that meets stakeholders' needs.

- **Software configuration management**

  manages the effects of change throughout the software process.

- **Reusability management**

  defines criteria for work product reuse (including software components) and establishes mechanisms to achieve reusable components.

- **Work product preparation and production**

  encompass the activities required to create work products such as models, documents, logs, forms, and lists.

Adapting a Process Model

- the overall flow of activities, actions, and tasks and the interdependencies among them
- the degree to which actions and tasks are defined within each framework activity
- the degree to which work products are identified and required
- the manner which quality assurance activities are applied
- the manner in which project tracking and control activities are applied
- the overall degree of detail and rigor with which the process is described
- the degree to which the customer and other stakeholders are involved with the project
- the level of autonomy given to the software team
- the degree to which team organization and roles are prescribed

The Essence of Practice in Software Engineering

- Polya suggests:
  1. *Understand the problem* (communication and analysis).
  2. *Plan a solution* (modeling and software design).
  3. *Carry out the plan* (code generation).
  4. *Examine the result for accuracy* (testing and quality assurance).

Understand the Problem

- *Who has a stake in the solution to the problem?* That is, who are the stakeholders?
- *What are the unknowns?* What data, functions, and features are required to properly solve the problem?
- *Can the problem be compartmentalized?* Is it possible to represent smaller problems that may be easier to understand?
- *Can the problem be represented graphically?* Can an analysis model be created?

Plan the Solution

- *Have you seen similar problems before?* Are there patterns that are recognizable in a potential solution? Is there existing software that implements the data, functions, and features that are required?
- *Has a similar problem been solved?* If so, are elements of the solution reusable?
- *Can subproblems be defined?* If so, are solutions readily apparent for the subproblems?
- *Can you represent a solution in a manner that leads to effective implementation?* Can a design model be created?

Carry Out the Plan

- *Does the solution conform to the plan?* Is source code traceable to the design model?
- *Is each component part of the solution provably correct?* Has the design and code been reviewed, or better, have correctness proofs been applied to algorithm?

Examine the Result

- *Is it possible to test each component part of the solution?* Has a reasonable testing strategy been implemented?
- *Does the solution produce results that conform to the data, functions, and features that are required?* Has the software been validated against all stakeholder requirements?

Hooker's General Principles

1: *The Reason It All Exists*

A software system exists for one reason: *to provide value to its users*

2: *Keep It Simple, Stupid!*

All design should be as simple as possible. This facilitates having a more easily understood and easily maintained system.

3: *Maintain the Vision*

4: *What You Produce, Others Will Consume*

5: *Be Open to the Future*

A system with a long lifetime has more value

6: *Plan Ahead for Reuse*

7: *Think!*

Placing clear, complete thought before action almost always produces better results

What is Software?

Software is:
(1) instructions (computer programs) that when executed provide desired features, function, and performance;
(2) data structures that enable the programs to adequately manipulate information and
(3) documentation that describes the operation and use of the programs.

The Nature of Software

- Software takes on a dual role. It is a **product**, and at the same time, the **vehicle for delivering a product**.
- As a product, it delivers the computing potential embodied by computer hardware or more broadly, by a network of computers that are accessible by local hardware.
- As the vehicle used to deliver the product, software acts as the basis for the control of the computer (operating systems), the communication of information (networks), and the creation and control of other programs.
- Software delivers the most important product of our time—information.

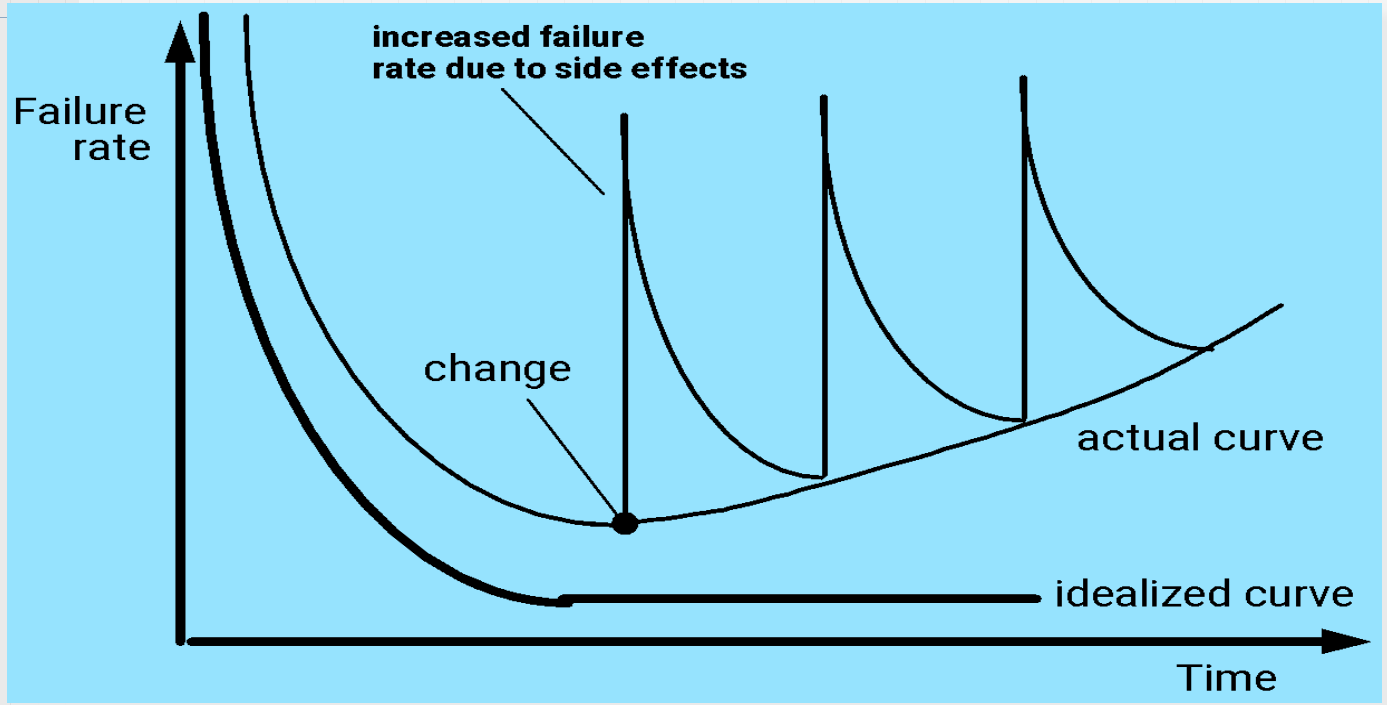
What is Software?

- *Software is developed or engineered, it is not manufactured in the classical sense.*
- *Software doesn't "wear out."*
- *Although the industry is moving toward component-based construction, most software continues to be custom-built.*

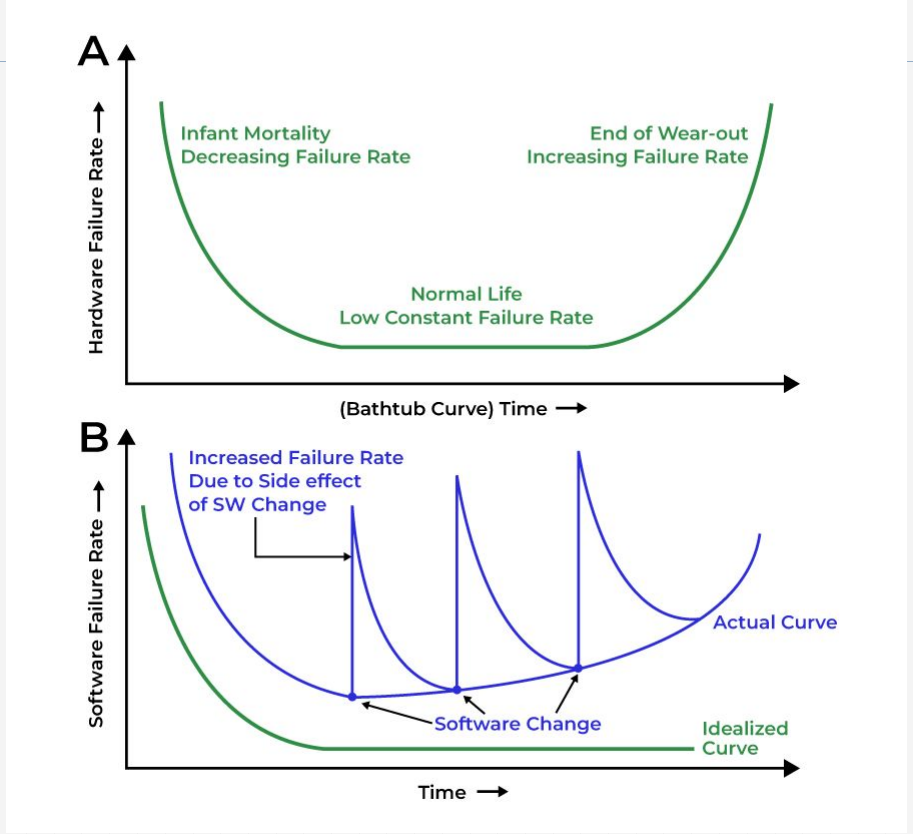
Software is developed or engineered, not manufactured

- Software development is an engineering process, not a manufacturing one, emphasizing design, creation, and adaptation rather than mass production of physical goods.
- Software development emphasizes the design and creation of logical systems, rather than the physical assembly of components like in manufacturing.
- While reusable components exist, most software remains custom-built to meet specific needs and requirements.

Wear vs. Deterioration



Failure Curve **h/w** vs **s/w**



Wear vs. Deterioration

- Software doesn't "**wear out**" like hardware in the physical sense, and there are no software "spare parts"

- software can deteriorate or become harder to maintain over time due to changes, accumulating technical debt, and other factors, a phenomenon sometimes called "software rot".

Technical Debt

- **Technical debt** is the cost of future rework or maintenance incurred when prioritizing speed and shortcuts over code quality and robust solutions.
- Technical debt arises when developers choose quick fixes or suboptimal solutions to meet deadlines or simplify development, knowing that these choices will create problems later.

Consequences:

- Increased development costs
- Reduced maintainability
- Performance issues

Legacy Systems/Software

- **Legacy systems/software** encompasses any software, application, or computer system that is considered outdated or obsolete, but is still being used by an organization.
- Legacy software poses several challenges, including high maintenance costs, security vulnerabilities, integration difficulties, and difficulty adapting to new technologies and business needs, which can hinder innovation and efficiency.

Legacy Software

Why must it change?

- software must be adapted to meet the needs of new computing environments or technology.
- software must be enhanced to implement new business requirements.
- software must be extended to make it interoperable with other more modern systems or databases.
- software must be re-architected to make it viable within a network environment.

Component based vs Custom Built

- While component-based development offers advantages like reusability and faster development, most software continues to be built using a custom approach, often with some glue logic to assemble pre-existing components, rather than being entirely composed of Commercially off-the-shelf components (COTS).
- Custom built software offers flexibility, tailored solutions, and potential for business improvements, but can be expensive and time-consuming.
- Component-based software enables faster development, reusability, and improved maintainability, but may require more planning and expertise.

Software Myths

- **Software myths**—common misconceptions about software development—that often lead to project failures or inefficiencies.
- These myths are categorized into three types: Management Myths, Customer Myths, and Practitioner Myths.

Management Myths

- **Myth**: *We already have a book of standards and procedures—why do we need more?*

  Reality: While documented standards are useful, software development is dynamic and requires continuous updates and improvements to methodologies.

___

- **Myth**: *If we get behind schedule, we can add more programmers and catch up.*

  Reality: Adding more people to a late project often increases complexity and further delays the project (as described in Brooks' Law).

___

- **Myth:** *Once we write the program and get it working, our job is done.*

  Reality: The majority of software costs come from maintenance, enhancements, and bug fixes after deployment.

___

- **Myth:** *We can outsource software development and avoid in-house expertise.*

  Reality: Even with outsourcing, organizations need internal knowledge to manage, validate, and maintain the software.

Customer Myths

- **Myth:** *A general statement of objectives is enough to get started. We can refine requirements later.*

  Reality: Vague requirements often lead to scope creep, misunderstandings, and cost overruns. Clear, well-defined requirements are essential from the start.

  ___

- **Myth:** *Software requirements keep changing, but change is easy since software is flexible.*

  Reality: Changes late in development can be expensive and time-consuming. Proper planning and change control processes are necessary.

Developer Myths

- **Myth**: *Once I write the code and it works, my job is done.*

  Reality: Code needs to be tested, reviewed, and maintained. Good documentation and code quality are critical for long-term success.

  ___

- **Myth**: *The only deliverable from a software project is the working program.*

  Reality: Software projects also need documentation, user manuals, maintenance plans, and testing reports.

  ___

- **Myth**: *Software engineering will make development slow and bureaucratic.*

  Reality: Proper software engineering practices improve efficiency, reduce errors, and prevent costly rework.

Software Myths

- Affect managers, customers (and other non-technical stakeholders) and practitioners
- Are believable because they often have elements of truth,

but ...

- Invariably lead to bad decisions,

therefore ...

- Insist on reality as you navigate your way through software engineering

Software Engineering

- Some realities:
  - *a concerted effort should be made to understand the problem before a software solution is developed*
  - *design becomes a pivotal activity*
  - *software should exhibit high quality*
  - *software should be maintainable*

- The seminal definition:
  - *[Software engineering is] the establishment and use of sound engineering principles in order to obtain economically software that is reliable and works efficiently on real machines.*

Case studies

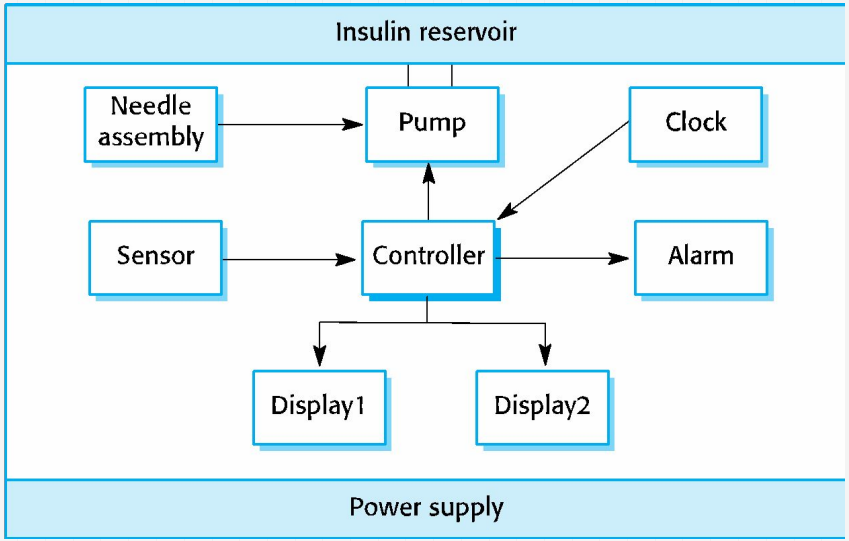
- **A personal insulin pump**

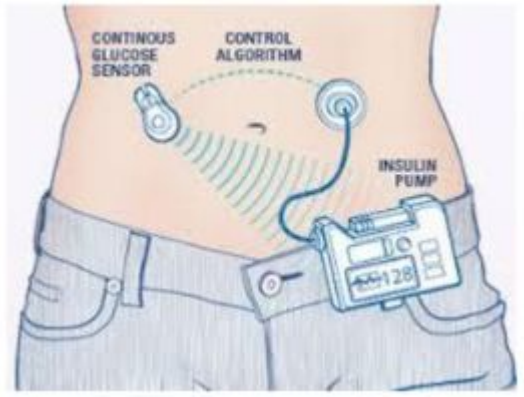
  An embedded system in an insulin pump used by diabetics to maintain blood glucose control.

Insulin pump control system

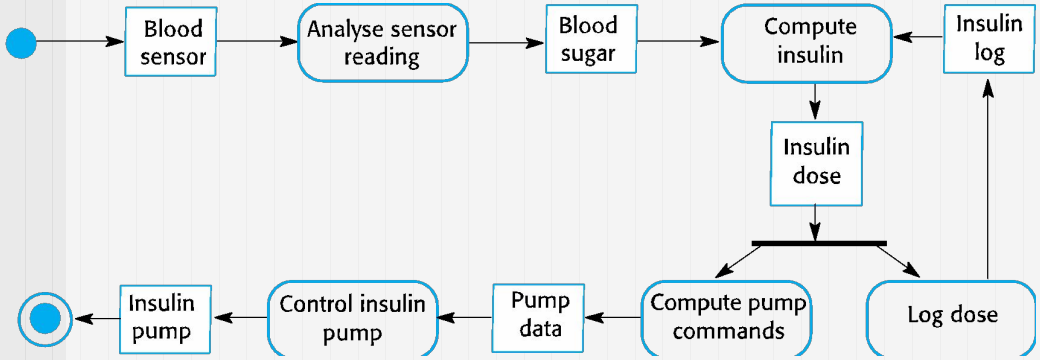
- Collects data from a blood sugar sensor and calculates the amount of insulin required to be injected.
- Calculation based on the rate of change of blood sugar levels.
- Sends signals to a micro-pump to deliver the correct dose of insulin.
- Safety-critical system as low blood sugars can lead to brain malfunctioning, coma and death; high-blood sugar levels have long-term consequences such as eye and kidney damage.

Insulin pump hardware architecture





Activity model of the insulin pump



Essential high-level requirements

- The system shall be available to deliver insulin when required.
- The system shall perform reliably and deliver the correct amount of insulin to counteract the current level of blood sugar.
- The system must therefore be designed and implemented to ensure that the system always meets these requirements.

References

Books

1. Software Engineering: Ian Sommerville, 10th Edition
2. Software Engineering: A Practitioner's Approach- Roger Pressman and Bruce Maxim, 9th Edition